

Programmeerimise eksami arvutiosa näidisülesanded 2023

Ülesanne 1. Saja Aakri mets (40 p)

Saja Aakri metsa elaniku karupoeg Puhhi fännidest metsaomanikud on oma metsatükkide pindalad kirjutanud [aakrites](#) (1 aaker = 0,4047 hektarit). Igal omanikul on ainult ühe puuliigi metsad. Konkreetsete puuliikide puhul on teada aastane metsa juurdekasv hektari kohta tihumeetrites (tm/ha). Näiteks kase puhul võib see olla 4,8 tm/ha, kuuse puhul 6,6 tm/ha, männi puhul 3,7 tm/ha. Omanik tahab teada, mitu tihumeetrit metsa aastas teatud suurusest suuremates metsatükkides juurde kasvab.

Koostada funktsioon juurdekasv, mis

- võtab argumentideks metsatüki pindala (ujukomaarv aakrites) ja metsa aastase juurdekasvu hektari kohta (ujukomaarv),
- tagastab selle pindalaga metsatüki aastase juurdekasvu ümardatuna sajandikeni.

Näide funktsiooni juurdekasv tööst

```
>>> juurdekasv(3.78, 6.6)
10.1
```

Koostada programm, mis

- küsib kasutajalt
 - failinime (failis on eraldi ridadel metsatükkide pindalad aakrites);
 - vastava puuliigi aastase juurdekasvu hektari kohta tihumeetrites (ujukomaarv);
 - piiri, mitmest aakrist suuremad metsatükid arvesse võtta (ujukomaarv);
- loeb failist metsatükkide pindalad;
- arvutab (funktsiooni juurdekasv abil) ja väljastab metsatüki aastase juurdekasvu, kui selle metsatüki pindala on sisestatud piirist suurem;
- väljastab teate "Metsatükki ei võeta arvesse", kui metsatüki pindala ei ole sisestatud piirist suurem;
- väljastab lõpuks ekraanile, mitme metsatüki juurdekasv arvutati.

Näide faili *andmed.txt* sisust

```
0.9
3.78
2.05
1.58
```

Näide programmi tööst (kasutaja sisend on paksus kirjas)

```
Sisestage failinimi: andmed.txt
Sisestage aastane juurdekasv hektari kohta tihumeetrites: 6.6
Sisestage piir, mitmest aakrist suuremad metsatükid arvesse võtta: 2
Metsatüki aastane juurdekasv on 10.1
Metsatüki aastane juurdekasv on 5.48
Metsatükki ei võeta arvesse
Arvutati 2 metsatüki juurdekasv
```

Ülesanne 2. Turniir (15 p)

Koosta programm turniiri läbiviimiseks. Osalejate nimed on failis *turniir.txt*. Esimesel real on tühikutega eraldatud voorude numbrid (esimese rea alguses on viis tühikut järjest) ja igal järgneval real on osaleja nimi ja voorude punktide arvud, mis on samuti eraldatud tühikutega. Kui osaleja ei ole mõnes voorus veel sooritus teinud, on numbri asemel kriips. Osalejate ja voorude arv ei ole ette teada, s.t programm peab töötama ka siis, kui osalejaid ja/või voore on vähem või rohkem.

Näide faili *turniir.txt* sisust

```
1 2 3 4 5 6
Mari 2 4 5 - 1 4
Juku 1 3 2 7 8 -
Malle - 2 - 3 2 5
Kalle 4 6 - - 2 -
```

Kirjuta funktsioon `loe_seis`, mis võtab argumendiks failinime ja tagastab sõnastiku, mille võtmeteks on turniiril osalejate nimed ja väärtusteks voorude tulemuste järjendid (punktid on järjendites andmetüübilt täisarvud).

Näide funktsiooni `loe_seis` tööst

```
>>> loe_seis("turniir.txt")
{'Mari': [2, 4, 5, '-', 1, 4], 'Juku': [1, 3, 2, 7, 8, '-'], 'Malle':
['-', 2, '-', 3, 2, 5], 'Kalle': [4, 6, '-', '-', 2, '-']}
```

Koosta funktsioon `lisa_tulemus` kindla vooru punktide salvestamiseks. Funktsioon võtab argumentideks osaleja nime, vooru järjekorranumbri (täisarv), sõnastiku ja lisatava tulemuse (täisarv). Kui sellel osalejal vastavas voorus veel tulemust ei ole, asendab funktsioon `lisa_tulemus` vastava kriipsu sõnastikus tulemusega ning väljastab teate, et tulemus on lisatud. Kui osalejal on selle vooru tulemus juba olemas, väljastab funktsioon selle kohta teate ja ei muuda midagi. Funktsioon tagastab sõnastiku uue seisuga.

Näide funktsiooni `lisa_tulemus` tööst eelpool toodud sõnastikuga

```
>>> lisa_tulemus("Mari", 3, sõnastik, 0)
Tulemus on juba varem lisatud!
{'Mari': [2, 4, 5, '-', 1, 4], 'Juku': [1, 3, 2, 7, 8, '-'], 'Malle':
['-', 2, '-', 3, 2, 5], 'Kalle': [4, 6, '-', '-', 2, '-']}
>>> lisa_tulemus("Mari", 4, sõnastik, 2)
Tulemus lisatud!
{'Mari': [2, 4, 5, 2, 1, 4], 'Juku': [1, 3, 2, 7, 8, '-'], 'Malle':
['-', 2, '-', 3, 2, 5], 'Kalle': [4, 6, '-', '-', 2, '-']}
```

Koosta funktsioon `leia_skoor`, mis võtab argumentideks osaleja nime ja sõnastiku ning tagastab selle osaleja punktisumma.

Näide funktsiooni `leia_skoor` tööst eelpool toodud sõnastikuga

```
>>> leia_skoor("Malle", sõnastik)
12
```

Koosta põhiprogramm, kus kasutaja saab valida erinevate tegevuste vahel. Pärast iga tegevust küsib programm uuesti, mida kasutaja soovib teha, kuni ta valib programmi töö lõpetamise. Kui kasutaja sisestab

- "1", kuvab programm kõigi osalejate tulemused (ühe inimese tulemused ühel real eraldatuna tühikutega)
- "2", lisab programm osaleja tulemuse järgmiste sammude abil:
 - küsib nime
 - küsib vooru numbrit
 - küsib tulemust
 - lisab tulemuse ja väljastab sellest teate, kasutades funktsiooni `lisa_tulemus`. Kui tulemus on juba olemas, väljastab funktsioon selle kohta teate.
- "3", küsib programm osaleja nime ja leiab funktsiooni `leia_skoor` kasutades selle osaleja skoori (punktisumma) ning väljastab tulemuse sobival kujul ekraanile (vt näiteid).
- "4", leiab programm suurima skooriga osaleja ja väljastab tema nime ja punktisumma
- "5", teeb programm järgmised sammud:
 - salvestab turniiritabeli uue seisu esialgse failiga samal kujul faili `turniir_uus.txt`
 - väljastab lõpetamise kohta teate
 - lõpetab töö

Näide programmi tööst (kasutaja sisend on paksus kirjas)

```
Vali tegevus:
1 - Vaata punkt tabelit
2 - Lisa tulemus
3 - Vaata skoori
4 - Leia võitja
5 - Lõpeta programmi töö
1
Mari 2 4 5 - 1 4
Juku 1 3 2 7 8 -
Malle - 2 - 3 2 5
Kalle 4 6 - - 2 -
Vali tegevus:
1 - Vaata punkt tabelit
2 - Lisa tulemus
3 - Vaata skoori
4 - Leia võitja
5 - Lõpeta programmi töö
2
Sisesta nimi: Mari
Sisesta voor: 4
Sisesta punktid: 2
Tulemus lisatud!
Vali tegevus:
```

- 1 - Vaata punktitalbelit
- 2 - Lisa tulemus
- 3 - Vaata skoori
- 4 - Leia võitja
- 5 - Lõpeta programmi töö

1

Mari 2 4 5 2 1 4

Juku 1 3 2 7 8 -

Malle - 2 - 3 2 5

Kalle 4 6 - - 2 -

Vali tegevus:

- 1 - Vaata punktitalbelit
- 2 - Lisa tulemus
- 3 - Vaata skoori
- 4 - Leia võitja
- 5 - Lõpeta programmi töö

3

Sisesta nimi: **Mari**

Mari skoor on 18.

Vali tegevus:

- 1 - Vaata punktitalbelit
- 2 - Lisa tulemus
- 3 - Vaata skoori
- 4 - Leia võitja
- 5 - Lõpeta programmi töö

4

Suurima skooriga on Juku (21 punkti).

Vali tegevus:

- 1 - Vaata punktitalbelit
- 2 - Lisa tulemus
- 3 - Vaata skoori
- 4 - Leia võitja
- 5 - Lõpeta programmi töö

5

Programm lõpetas töö.

Pärast programmi töö lõppu peaks faili *turniir_uus.txt* sisu olema:

```
1 2 3 4 5 6
Mari 2 4 5 2 1 4
Juku 1 3 2 7 8 -
Malle - 2 - 3 2 5
Kalle 4 6 - - 2 -
```

Ülesanne 3. Tondid Tartus (5 p)

Loo klass nimega `Tont`, millel on isendiväljad `nimi` (sõne), `vanus` (täisarv) ja `elukoht` (sõne). Loo klassi konstruktor, mis võtab argumendiks nime, vanuse ja elukoha ning määrab vastavad isendiväljade väärtused. Lisaks loo klassile `Tont` isendimeetod `kummita`, mis ei võta argumente ja väljastab ekraanile kummitamise tondi nimega ja tema elukohaga. Samuti lisa klassile meetod `__str__`, mis tagastab isendiväljade info kujul "Nimi: [nimi], vanus: [vanus], elukoht: [elukoht]".

Lisaks loo klassile `Tont` alamklass `Võlur`, millel on lisaks isendimeetod `nõiu`, mis võtab argumendiks nõitutava **isendi** ja väljastab nõidumist sisaldava sõne võluri nimega ja nõiduse saanud isendi nimega.

Lõpuks loo kolm tonti:

1. `tont` nimega Norbert, 31-aastane, elukoht Tartu;
2. `võlur` nimega Harry, 17-aastane, elukoht Tartu;
3. `võlur` nimega Snape, 35-aastane, elukoht Tartu.

Loo põhiprogramm, kus ekraanile kuvatakse tontide info, kus Norbert kummitab ja Harry nõiub Snape'i.

Näide programmi tööst

```
Nimi: Norbert, vanus: 31, elukoht: Tartu
Norbert kummitab elukohas Tartu!
Nimi: Harry, vanus: 17, elukoht: Tartu
Nimi: Snape, vanus: 35, elukoht: Tartu
Harry pani nõiduse, millega sai pihta Snape!
```